TECHNICAL REPORT

API-Based Weather Analytics Dashboard

Live Data Acquisition, Transformation & Visualisation for Leipzig, Germany using Python and Power BI

Prepared by

Sumanth Pidishetti (100008679)

Sumanth Gadwala (100008777)

Rohith Reddy Rudraiah Gari (100008638)

Danush Kumar Sekar (100008245)

Course: Advanced Programming - Python

Location of Study: Leipzig, Germany

Date: 17 June 2026

Table of Contents

1. Executive Summary

This technical report outlines the API-Based Weather Analytics Dashboard, which demonstrates a working pipeline from live public REST API data to an interactive dashboard focused on Leipzig, Germany. The project automates the retrieval, transformation, and visualization of a seven-day, hourly weather and air-quality forecast.

The architecture consists of a Python-based data acquisition and transformation process, direct integration of processed data into Power BI, and a visual dashboard with interactive views for temperature, forecasts, and air quality.

Key features include live data collection, robust data transformation, a modular program structure, and a fully interactive dashboard. The system also incorporates air-quality metrics for comprehensive environmental analysis.

Each execution generates a dataset of 168 hourly records, each with twelve fields covering weather and air-quality metrics. The report details the problem context, technical design, implementation, results, and recommendations for future improvements.

2. Introduction

2.1 Background and Motivation

Weather data is one of the most universally accessible categories of live, public API data, making it a natural subject for a project intended to demonstrate end-to-end data pipeline construction. Unlike static datasets, a weather feed changes continuously, which forces the pipeline to be genuinely live rather than a one-time extraction exercise. It also naturally exhibits the kind of structural complexity — deeply nested JSON, optional fields, and mixed data types — that mirrors real-world data engineering problems far more closely than a flat CSV file would.

Leipzig was selected as the subject city both for its relevance to the team and because a single-city scope keeps the dataset small enough to inspect and validate by hand at every stage of the pipeline, while still being representative of the kind of multi-day, multi-metric forecasting data used in production weather applications.

2.2 Project Objectives

The project was scoped around four explicit technical objectives, each of which maps to a deliverable component of the final system:

1. Establish a live connection to a public REST API and retrieve structured forecast data using HTTP requests.

2. Transform the raw, deeply nested JSON response into a clean, typed, tabular dataset suitable for analytical tooling.
3. Organise the implementation into a modular program structure, with each stage of the pipeline isolated into its own logical section.
4. Connect the transformed dataset directly to an interactive dashboard, enabling exploration of the data without manual export or reformatting steps.

A secondary objective, adopted by the team to extend the project beyond minimum requirements, was to incorporate air-quality analytics alongside the core weather metrics, since the chosen API exposes this data through a single additional query parameter.

2.3 Scope

The scope of the project covers a single city (Leipzig), a seven-day forecast horizon at hourly resolution, and a fixed set of twelve output fields. The system is designed to be re-run on demand — each execution of the Python script and subsequent Power BI refresh pulls the latest forecast available from the API at that moment, rather than operating against a fixed historical snapshot. Historical data retention, multi-city comparison, and automated scheduling were treated as candidates for future work rather than requirements of the current scope, and are discussed in Section 9.

3. Technology Stack

The project is built on four complementary technologies, each responsible for a distinct stage of the pipeline. Table 1 summarises their roles, and the subsections that follow provide further detail on why each was selected.

Table 1. Technology stack and corresponding pipeline role

Technology	Role in Pipeline
Python 3	Core scripting language; orchestrates API calls, JSON parsing, and data transformation.
WeatherAPI.com	External REST API data source; free tier; supplies 7-day hourly forecast and air quality data.
Pandas	Builds the structured DataFrame, applies type conversion, and parses datetime values.
Power BI Desktop	Consumes the Pandas DataFrame as a native Python data source; powers the interactive dashboard.

3.1 Python

Python was chosen as the orchestration language because of its concise HTTP tooling (the requests library) and its first-class support for JSON, both of which keep the data-acquisition code compact and readable. Python is also the only one of the four technologies capable of directly integrating with Power BI's native scripting data source, which allows the entire transformation pipeline to run without any intermediate file format.

3.2 WeatherAPI.com

WeatherAPI.com was selected as the data source because its free tier supports a seven-day hourly forecast horizon and, critically, an optional air-quality parameter that returns four distinct pollution indices without requiring a second API or paid subscription. Its JSON response structure, while nested, follows a predictable and well-documented schema, which made it a practical choice for an assignment with a fixed timeline.

3.3 Pandas

Pandas serves as the transformation layer between the raw API response and the dashboard. Its DataFrame constructor accepts a list of dictionaries directly, which maps naturally onto the per-hour records produced during JSON parsing. Pandas' to_datetime function also resolves a recurring practical problem in weather data integration: ensuring that timestamp strings are converted into a true datetime type that downstream tools can use as a time axis, rather than being left as plain text.

3.4 Power BI

Power BI was chosen as the visualisation layer primarily because of its native Python Script data source, which allows a complete Python script to be embedded directly inside the Power Query engine. This eliminates the need to export the DataFrame to a CSV file or a database table before visualising it, keeping the entire pipeline as a single artefact that can be rerun end-to-end.

4. System Architecture and Data Flow

The system follows a linear, five-stage pipeline, illustrated conceptually in Table 2. Each stage consumes the output of the previous stage and is implemented as an isolated section of the Python script, thereby satisfying the modular program structure requirement discussed further in Section 6.

Table 2. End-to-end data flow

Stage	Description
1. Source	WeatherAPI.com REST endpoint, queried with city, forecast length, and air-quality flag.
2. Fetch	Python's requests.get() issues the HTTP GET request; response.json() decodes the payload.
3. Parse	A nested loop iterates over the forecast day and hour structures, extracting fields using safe .get() calls.
4. Transform	Parsed records are loaded into a Pandas DataFrame; the DateTime column is cast to a true datetime type.
5. Visualise	Power BI's Python Script data source loads df_master directly; Power Query applies final shaping steps.

A defining characteristic of this architecture is that it is stateless and refresh-driven: no stage persists data between runs, and the entire chain — from HTTP request to rendered dashboard — re-executes whenever the Power BI report is refreshed. This means the dashboard always reflects the most recent forecast available from the provider at the moment of refresh, rather than a cached or historical snapshot.

The output volume at the end of Stage 4 is fixed by the API's contract: seven days multiplied by twenty-four hours produces exactly 168 records, and the twelve fields extracted per record (described fully in Section 5.3) are consistent across every run, which makes the downstream Power BI data model stable even though the underlying values change with each refresh.

5. Implementation Detail

5.1 API Configuration and Data Fetch

The first two stages of the pipeline are responsible for constructing the request and retrieving the raw response. The API key, target city, and endpoint are defined as simple variables. The request URL is assembled using an f-string that embeds two query parameters of particular importance: days=7, which requests the maximum forecast horizon available on the free tier, and aqi=yes, which instructs the API to include the nested air-quality block for every hourly record. Listing 1 shows the relevant code.

Listing 1. API configuration and request

```
import requests
import pandas as pd

# 1. API Configuration
api_key = "ffb5b62e059b4672b49130304260406"
city = "Leipzig"

# 2. Fetch live 7-day forecast including Air Quality
url =
f"http://api.weatherapi.com/v1/forecast.json?key={api_key}&q={city}&days=7&aqi=yes"
response = requests.get(url)
data = response.json()
```

The choice to pass the city name as a plain string rather than coordinates keeps the configuration human-readable, at the minor cost of relying on the API provider's internal geocoding to resolve "Leipzig" to the correct location. For a single, unambiguous city name, this introduces negligible risk. However, it is a consideration that would need to be revisited if the system were extended to support cities with duplicate names across different countries, as discussed in Section 9.

5.2 Defensive JSON Parsing

The third stage is the most structurally complex part of the pipeline because the API response nests three levels deep: a top-level forecast object contains a list of forecastday objects, each of which contains a list of 24-hour objects. A double loop is used to walk this structure, and within the innermost loop, every field is read using the dictionary .get() method rather than direct key indexing.

This defensive pattern is applied for two reasons. First, the air_quality block itself is read with a default empty dictionary — hour.get('air_quality', {}) — which means that if the API ever omits air-quality data for a particular hour (for example, due to a temporary upstream data gap), the script does not raise a KeyError; it simply returns an empty dictionary, and any subsequent .get() call against that empty dictionary safely returns None rather than crashing. Second, applying the same .get() pattern to every other field protects the pipeline against any single missing attribute anywhere in the payload, which is a realistic possibility in a third-party API that aggregates data from multiple upstream meteorological sources.

Listing 2. Nested JSON parsing with defensive field access

```
# 3. Loop through the JSON structure safely using .get()
hourly_data = []

for day in data['forecast']['forecastday']:
    for hour in day['hour']:
        # Extract the air quality dictionary safely
        aqi_data = hour.get('air_quality', {})
```

```
        hourly_data.append({
            'DateTime': hour.get('time'),
            'Temperature (°C)': hour.get('temp_c'),
            'Humidity (%)': hour.get('humidity'),
            'Precipitation (mm)': hour.get('precip_mm'),
            'Feels Like (°C)': hour.get('feelslike_c'),
            'Wind Speed (km/h)': hour.get('wind_kph'),
            'Cloud Cover (%)': hour.get('cloud'),
            'Chance of Rain (%)': hour.get('chance_of_rain'),

            # Safely grab AQI metrics; missing values return None
            'PM2.5 (Fine Particulates)': aqi_data.get('pm2_5'),
            'PM10 (Particulates)': aqi_data.get('pm10'),
            'US EPA Index': aqi_data.get('us-epa-index'),
            'European AQI': aqi_data.get('gb-defra-index'
        })
```

Each iteration of the inner loop appends exactly one dictionary to the hourly_data list, with twelve named keys. Because the outer loop runs seven times (once per forecast day) and the inner loop runs twenty-four times (once per hour), the list reaches exactly 168 entries by the time parsing completes, with no further bookkeeping required.

5.3 Transformation into a Structured DataFrame

The fourth stage converts the flat list of dictionaries into a Pandas DataFrame and performs one critical type correction. Listing 3 shows the two lines of code responsible for this stage.

Listing 3. DataFrame construction and datetime normalisation

```
# 4. Convert to Pandas DataFrame and enforce the date format
df_master = pd.DataFrame(hourly_data)
df_master['DateTime'] = pd.to_datetime(df_master['DateTime'])
```

Because pd.DataFrame() accepts a list of uniformly structured dictionaries directly, no manual column naming or reshaping is required at this step. Pandas infers all twelve columns from the dictionary keys used during parsing. The second line is nevertheless essential: WeatherAPI.com returns its time field as a plain string, and without an explicit conversion to a true datetime64 type, Power BI would treat the column as text, preventing it from being used as a continuous time axis on the dashboard's line charts. Table 3 lists the resulting schema.

Table 3. Final df_master schema (12 columns, 168 rows)

Column	Type	Description
DateTime	datetime64	Timestamp for the hourly record; used as the time axis for all charts.
Temperature (°C)	float	Hourly air temperature.
Humidity (%)	int	Relative humidity.

Precipitation (mm)	float	Rainfall amount for the hour.
Feels Like (°C)	float	Apparent temperature accounting for wind and humidity.
Wind Speed (km/h)	float	Wind speed.
Cloud Cover (%)	int	Percentage of sky covered by cloud.
Chance of Rain (%)	int	Forecast probability of precipitation.
PM2.5 (Fine Particulates)	float	Fine particulate matter under 2.5 micrometres.
PM10 (Particulates)	float	Coarse particulate matter under 10 micrometres.
US EPA Index	int	US EPA air quality scale (1–6).
European AQI	int	UK DEFRA-based air quality index.

6. Modular Program Structure

One of the project's explicit technical requirements was a modular program structure, and this was addressed by dividing the entire pipeline into five clearly delineated sections within the same script, each carrying a single, self-contained responsibility. Table 4 summarises the five sections as implemented.

Table 4. Modular sections of the pipeline script

#	Section	Responsibility
1	API Configuration	Defines the API key, target city, and constructs the request URL with query parameters.
2	Data Fetch	Issues the HTTP GET request and decodes the JSON response.
3	JSON Parsing	Iterates over the nested forecast structure and defensively extracts fields into a flat list.
4	Transformation	Builds the Pandas DataFrame and normalises the DateTime column type.
5	Output	Exposes df_master as the variable consumed directly by Power BI's Python data source.

This separation of concerns offers a concrete, practical benefit beyond satisfying the assignment criterion: each section can be modified independently without risk to the others. For example, switching to a different weather provider would require changes only to Section 1 (and possibly the field names used in Section 3). In contrast, the transformation and output logic in Sections 4 and 5 would remain untouched. Similarly, adding a new derived column to the dashboard would only require a new key in the dictionary built in Section 3, with no changes needed elsewhere in the script.

7. Power BI Integration

7.1 Connecting Python to Power BI

Power BI Desktop provides a native Python Script data source under its Get Data menu. Once selected, the entire Python script — spanning all five sections described in Section 6 — is pasted directly into the dialogue Power BI presents. When the query runs, Power BI's embedded Python engine executes the script in full and then inspects the resulting variable namespace for any DataFrame objects; df_master is automatically surfaced as a selectable table in the Navigator window.

This mechanism allows the pipeline to avoid exporting any intermediate files. There is no CSV write step and no manual import; the DataFrame produced in memory by the Python interpreter is handed directly to Power Query. The five steps below summarise the connection workflow as performed for this project.

1. Open Power BI Desktop and navigate to Home → Get Data → More.
2. Select "Python script" from the list of available data source types.
3. Paste the complete pipeline script (Sections 1–5) into the script editor and confirm.
4. In the Navigator window, select the df_master table to load it into the data model.
5. Apply final shaping steps inside Power Query, as detailed in Section 7.2.

7.2 Power Query Applied Steps

After df_master is loaded, Power Query records a sequence of Applied Steps that further refine the table before it reaches the report canvas. For this project, the steps recorded were: Source, Navigation, Changed Type, Inserted Date, Inserted Merged Column, Changed Type (second pass), and Renamed Columns. The underlying formula bar reference for the loaded table takes the form Source{[Name="df_master"]}[Value], which is Power Query's standard syntax for selecting a named object returned by a script-based data source.

The "Inserted Date" and "Inserted Merged Column" steps were used to derive a standalone date field from the DateTime column, which simplifies grouping the 168 hourly records into the seven daily aggregates shown in the forecast table visual. The two "Changed Type" steps ensure that all numeric columns are explicitly typed as decimal or whole numbers rather than being left as the generic "Any" type that Power Query sometimes assigns to data sourced from a script.

8. Dashboard Design and Key Visualisations

The resulting Power BI report is organised around a single page containing four coordinated visuals plus a set of toggle controls, designed so that a viewer can move from a high-level current-conditions summary down to hour-by-hour and metric-specific detail without leaving the page.

8.1 Current Weather Summary

In the upper-left of the report, a card-style visual displays the current temperature alongside the “feels like” value, current wind speed, humidity, and a coloured status badge summarising the prevailing air-quality category (for example, “Good”). This visual is intended as the first thing a viewer reads, answering the single most common question — “what is it like right now” — before any further exploration.

8.2 Seven-Day Forecast Table

Adjacent to the current-conditions card, a tabular visual lists each of the next several days together with average temperature, average humidity, and the day’s chance of rain. This view is driven by the date field produced through the “Inserted Date” step described in Section 7.2, and gives a forward-looking complement to the current-conditions card.

8.3 24-Hour Temperature Comparison

A dual-line chart plots Temperature and feels-like temperature across all twenty-four hours of the selected day, using the DateTime column as its time axis. This visual exposes the divergence between actual and apparent temperature, which is most pronounced in the early afternoon, when wind and humidity effects on perceived temperature are typically strongest.

8.4 Hourly Weather Strip

A horizontal strip visual presents temperature values at regular intervals across the day, giving an at-a-glance read of the day’s overall shape — for example, making a rainy afternoon or a sharp evening cool-down immediately visible without requiring the viewer to read the line chart in detail.

8.5 Toggle Views

Three toggle buttons allow the viewer to switch the report’s secondary panel between three distinct perspectives: the Seven-Day Forecast view described above, a PM10 versus PM2.5 comparison chart that plots the two particulate metrics against each other across the week, and an Average Cloud Cover view. This toggle mechanism is what elevates the report from a static set of charts to the interactive dashboard required by the assignment, since the underlying data model does not change — only the visual selected for display changes.

9. Air Quality Analytics Module

Air-quality analytics were incorporated as an extension beyond the core weather-tracking requirement by appending a single query parameter, aqi=yes, to the API request. This single change causes WeatherAPI.com to nest an additional air_quality object inside every hourly record, from which four

metrics are extracted: PM2.5 and PM10 particulate concentrations, the US EPA air-quality index, and a European AQI value sourced from the UK DEFRA index.

These four fields are processed through the same defensive .get() pattern used for the core weather fields, which means the air-quality module introduces no additional risk of pipeline failure even though it depends on a nested sub-object that the API does not guarantee to populate for every hour. In the dashboard, this data powers the PM10-versus-PM2.5 toggle view described in Section 8.5, adding an environmental dimension that complements the project's primary meteorological focus.

10. Challenges and Solutions

Four challenges of practical significance arose during implementation. Each is summarised in Table 5 together with the solution adopted.

Table 5. Challenges encountered and corresponding solutions

Challenge	Solution
Three-level-deep nested JSON structure (forecast → forecastday → hour).	A double loop iterates forecastday then hour, with field extraction isolated to the innermost loop body.
Air-quality sub-object occasionally absent, risking a KeyError.	hour.get('air_quality', {}) supplies an empty-dictionary default, so downstream .get() calls return None instead of raising.
Inconsistent or string-typed datetime values from the API.	pd.to_datetime() is applied immediately after DataFrame construction to enforce a true datetime64 column.
Passing Python output into Power BI without an intermediate export step.	Power BI's native Python Script data source runs the full script and exposes df_master directly to Power Query.

Of these, the defensive parsing pattern described for the second challenge is the one with the broadest effect on overall pipeline reliability: because the same .get()-with-default approach is applied uniformly across every field in Section 3 of the script, the pipeline degrades gracefully (producing a None value for an individual field) rather than failing whenever any single attribute is missing from the API response.

11. Results and Outcomes

The completed pipeline satisfies all four of the assignment's stated technical requirements and produces a working, interactive dashboard. Table 6 summarises the quantitative and qualitative outcomes of the project.

Table 6. Summary of project outcomes

Outcome	Detail
Live 7-day hourly forecast	168 records generated per run, refreshed automatically on every script execution.
Structured dataset	12 typed columns covering temperature, humidity, wind, precipitation, and 4 air-quality metrics.
Interactive dashboard	Single Power BI report page with 3 toggle-driven secondary views.
Requirements satisfied	HTTP requests, data transformation, modular structure, and live API integration — 4 of 4 criteria met.
Air quality integration	PM2.5, PM10, US EPA Index, and European AQI captured via a single additional query parameter.
Code organisation	5 independently modifiable sections, supporting straightforward extension or provider substitution.

Beyond the measurable outcomes in Table 6, the project also validated, in practice, a pattern that is broadly reusable beyond weather data: any REST API returning nested JSON can be connected to Power BI through the same five-stage structure — configure, fetch, parse defensively, transform with Pandas, and expose the resulting DataFrame as the script's output — without requiring a database, file export, or separate ETL tool.

12. Conclusion and Future Scope

12.1 Conclusion

This project successfully delivers a complete, modular data pipeline that connects a live public REST API to an interactive Power BI dashboard via a Python and Pandas transformation layer, fulfilling all stated assignment requirements while also extending the dashboard’s analytical scope to include environmental air-quality metrics. The defensive parsing strategy adopted throughout Section 3 of the script proved to be the single most valuable design decision, as it allows the pipeline to tolerate partial data gaps that are common when relying on a third-party aggregation API.

12.2 Future Scope

Several extensions were identified as natural next steps beyond the current scope of the project:

- Multi-city support, implemented via a dropdown selector in the dashboard that re-parameterises the city variable in Section 1 of the script, enabling side-by-side comparison across locations.

- Historical data retention, achieved by appending each run's output to a persistent CSV file or lightweight database rather than overwriting it, enabling trend analysis across weeks or months rather than the current seven-day rolling window.
- Automated, scheduled execution combined with an email-based summary report, removing the need for a manual Power BI refresh to keep the dashboard current.
- A finance-dashboard variant of the same architecture, substituting a stock or foreign-exchange API for WeatherAPI.com while reusing the fetch, parse, and transform stages largely unchanged.
- Deployment as a standalone web application using a framework such as Streamlit or Flask, broadening access beyond users with a Power BI Desktop installation.

13. References

WeatherAPI.com. *Weather API Documentation.* Retrieved from https://www.weatherapi.com/docs/

Pandas Development Team. *pandas documentation.* Retrieved from https://pandas.pydata.org/docs/

Microsoft. *Power BI Python script as a data source.* Retrieved from https://learn.microsoft.com/power-bi/

Python Software Foundation. *requests library documentation.* Retrieved from https://requests.readthedocs.io/